

Document number: P1951R0
Date: 2019-11-17
Audience: LWG
Reply-to: Logan R. Smith <logan.r.smith0@gmail.com>

Default Arguments for pair's Forwarding Constructor

[Introduction](#)

[Motivation](#)

[Impact](#)

[Proposed Changes](#)

[Acknowledgements](#)

Introduction

This paper proposes defaulting the template arguments `U1` and `U2` in `pair`'s forwarding constructor to `T1` and `T2` respectively, so that braced initializers may be used as constructor arguments to it.

Motivation

Consider this innocent-looking construction of a `std::pair`:

```
std::pair<std::string, std::vector<std::string>> p("hello", {});
```

This code uses simple, highly natural syntax for the constructor arguments; it is what any C++ programmer, beginner or expert, would hope to be able to write. During constructor overload resolution, two two-argument constructors (or constructor templates) are considered:

(A) *EXPLICIT* constexpr pair(const T1& x, const T2& y);

and

(B) template<class U1, class U2> *EXPLICIT* constexpr pair(U1&& x, U2&& y);

The more efficient option, and the one the user likely hoped would be used, is (B), which takes two forwarding references and perfectly forwards them to the constructors of `first` and `second`. However, since the second argument to the constructor was given as `{}`, the type of `U2` cannot be deduced, and so (B) is removed from overload resolution and (A) is selected. From there, a temporary `std::string` and `std::vector<std::string>` are created at the call site, and are passed by const reference to be copied into `first` and `second`. Thus, the simplest and easiest code to write results in potentially very inefficient behavior.

(Note, by contrast, if the `pair` is constructed using the similar-in-spirit `p("hello", std::vector<std::string>{})`, then (B) is selected, since `U2` can be deduced in this case. This subtlety of syntax is surprising and user-unfriendly.)

If (B)'s template arguments were adjusted slightly to default to the `pair`'s first and second type, respectively, there would be a fallback when deduction of braced initializers fails. Using the following adjusted signature:

(C) template<class U1=T1, class U2=T2> *EXPLICIT* constexpr pair(U1&& x, U2&& y);

this overload can now be selected for the example case above, so the example pair's string member is constructed in-place from a perfectly-forwarded string literal, and its vector member is move-constructed instead of copy-constructed from the temporary vector at the call site.

There is precedent in the standard library for using default template arguments specifically to accommodate braced initializers; for instance, `std::optional`'s forwarding constructor, and the second parameter of `std::exchange`. This paper recommends `std::pair` adopt this same strategy, for reducing surprise and making the most natural syntax be acceptably efficient.

Impact

This change would alter the meaning of existing code that uses braced initializers in the construction of pairs, likely changing copies to moves or perfectly-forwarded constructions in many cases. It is all but certain that this new behavior would be welcomed over the old.

Note that this proposal has no effect on APIs such as `std::map::emplace` which forward constructor arguments to `pair`, since those APIs need to deduce all argument types on their own first before forwarding them. Code such as

```
std::map<std::string, std::vector<std::string>> m;
m.emplace("hello", {});
```

remains ill-formed after this proposal.

Proposed Changes

Change the constructor's declaration in the synopsis in 23.4.2 [pairs.pair]:

```
template<class U1, class U2> EXPLICIT constexpr pair(U1&& x, U2&& y);
template<class U1=T1, class U2=T2> EXPLICIT constexpr pair(U1&& x, U2&& y);
```

Likewise, change the declaration around 23.4.2 [pairs.pair] p7:

```
template<class U1, class U2> EXPLICIT constexpr pair(U1&& x, U2&& y);
template<class U1=T1, class U2=T2> EXPLICIT constexpr pair(U1&& x, U2&& y);
```

Acknowledgements

Thanks to Narut Sereewattanawoot for helping discuss this problem and work out this solution.
